UNIVERSITÉ ALIOUNE DIOP DE BAMBEY
UFR Sciences & Technologies | Master 2 Big Data & Intelligence Artificielle

PROJET TUTORÉ — VERSION 2 CORRIGÉE

Système d'Alerte Précoce et Suivi Épidémiologique

Sentinel-SN

Durée	4 semaines
Équipe	2 étudiants
Domaine	Health Data Engineering, Streaming & Alerte Précoce
Version	v2 — Bugs corrigés, prérequis ajoutés, barème détaillé
Enseignant	Mr Ahmed Ben Sidy Bouya SEYE — Senior Big Data & AI Engineer | Groupe Sonatel

NiFi • Kafka • Spark Streaming • Pandera • HBase • Hive • Airflow • Privacy by Design

1. Prérequis & Installation

> ℹ️ **Configuration minimale recommandée**
> RAM : 16 Go minimum (Spark + Kafka + HBase + Airflow simultanés). CPU : 4 cœurs. OS : Ubuntu 22.04 LTS ou macOS 13+. Docker Engine 24.x + Docker Compose 2.x.

1.1 Vérification de l'environnement

📄 **check_env.sh**

```
# Vérifier Java 11 (requis par Spark et HBase)
java -version
# Attendu : openjdk version "11.0.x"

# Vérifier Docker et Docker Compose
docker --version       # Docker 24.x minimum
docker compose version # Docker Compose 2.x minimum

# Vérifier la RAM disponible
free -h
# Minimum : 14G disponibles

# Configurer les limites mémoire pour Elasticsearch/HBase
echo 'vm.max_map_count=262144' | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

1.2 Installation des dépendances Python

📄 **requirements.txt**

```
# requirements.txt — Sentinel-SN
# Testé avec Python 3.9 / Java 11 / Spark 3.3

# Core pipeline
pyspark==3.3.2
kafka-python==2.0.2

# Validation
pandera==0.17.2
pandas==2.0.3

# MLOps
apache-airflow==2.7.3
mlflow==2.8.1

# Stockage / Requêtes
pyhive==0.7.0
thrift==0.16.0
happybase==1.2.0        # Client HBase Thrift

# Analyse & Visualisation
matplotlib==3.7.3
seaborn==0.12.2
scipy==1.11.4
numpy==1.24.4

# Installation :
# pip install -r requirements.txt
# Java 11 requis : sudo apt install openjdk-11-jdk
```

📄 **install.sh**

```
# Créer un environnement virtuel (recommandé)
```

```
python3.9 -m venv venv_sentinel
source venv_sentinel/bin/activate

# Installer les dépendances
pip install -r requirements.txt

# Vérifier PySpark
python -c "import pyspark; print(pyspark.__version__)"
# Attendu : 3.3.2
```

1.3 Démarrage de l'infrastructure

start.sh

```
# Cloner le projet
git clone https://github.com/uadb/sentinel-sn.git && cd sentinel-sn

# Créer les répertoires de données
mkdir -p data/input nifi_templates dags models

# Démarrer tous les services (ordre important)
docker compose up -d zookeeper
sleep 10
docker compose up -d kafka nifi hbase hive-metastore
sleep 30
docker compose up -d spark-master spark-worker airflow

# Vérifier que tous les services sont UP
docker compose ps

# Initialiser les tables HBase (à exécuter UNE SEULE FOIS)
python hbase_setup.py

# Accès aux interfaces web :
#   NiFi    : http://localhost:8081
#   Spark   : http://localhost:8080
#   HBase   : http://localhost:16010
#   Airflow : http://localhost:8082 (admin/admin)
```

2. Codes Sources Corrigés

2.1 Initialisation HBase — hbase_setup.py

> ⚠️ **Nouveau fichier**
> Sans lui, le pipeline Spark ne peut pas écrire dans HBase. À exécuter une seule fois après docker compose up.

📄 **hbase_setup.py**

```
# hbase_setup.py — Création des tables HBase pour Sentinel-SN
# CORRECTION : les tables HBase doivent être créées avant le pipeline

import happybase
import logging

logger = logging.getLogger('HBaseSetup')

def create_sentinel_tables():
    """Crée les tables HBase nécessaires au pipeline Sentinel-SN."""
    conn = happybase.Connection('hbase', port=9090, timeout=10000)
    conn.open()

    tables_a_creer = {
        # Table principale : cas épidémiques anonymisés
        b'epidemio:cas': {
            b'info':    {'max_versions': 1},  # patient_id_secure, maladie_code
            b'stats':   {'max_versions': 5},  # nombre_cas, taux_incidence
            b'alerte':  {'max_versions': 1},  # statut_alerte, ingestion_ts
        },
        # Table alertes temps réel (TTL 7 jours)
        b'epidemio:alertes': {
            b'meta':    {'max_versions': 1, 'time_to_live': 604800},
            b'payload': {'max_versions': 1, 'time_to_live': 604800},
        },
        # Table seuils par district (mise à jour par Airflow)
        b'epidemio:seuils': {
            b'seuil':   {'max_versions': 10},  # historique des seuils
        },
    }

    tables_existantes = [t.decode() for t in conn.tables()]

    for table_name, families in tables_a_creer.items():
        nom = table_name.decode()
        if nom in tables_existantes:
            logger.info(f'Table {nom} déjà existante — ignorée')
        else:
            conn.create_table(table_name, families)
            logger.info(f'Table {nom} créée avec succès')

    conn.close()
    logger.info('Setup HBase Sentinel-SN terminé')

if __name__ == '__main__':
    logging.basicConfig(level=logging.INFO)
    create_sentinel_tables()
```

2.2 Contrats Pandera — schema.py

📄 **schema.py**

```
# schema.py — Contrats Pandera pour Sentinel-SN
```

```
import pandera as pa
from pandera.typing import Series
import pandas as pd
import logging

logger = logging.getLogger('SentinelSchema')

class EpidemicSchema(pa.SchemaModel):
    district_id:       Series[int] = pa.Field(ge=1, le=100)
    maladie_code:      Series[str] =
pa.Field(isin=['PALUD','DENGUE','CHOLERA','TYPHOIDE'])
    nombre_cas:        Series[int] = pa.Field(ge=0)
    nombre_deces:      Series[int] = pa.Field(ge=0)
    date_declaration:  Series[str] = pa.Field(str_matches=r'^\d{4}-\d{2}-\d{2}$')
    source_rapport:    Series[str] = pa.Field(isin=['DISTRICT','HOPITAL','API'])

    @pa.check('nombre_deces')
    def deces_inferieur_cas(cls, series, nombre_cas):
        return series <= nombre_cas

    class Config:
        strict = True
        coerce = True

class MeteoSchema(pa.SchemaModel):
    district_id:      Series[int]   = pa.Field(ge=1, le=100)
    date_mesure:      Series[str]   = pa.Field(str_matches=r'^\d{4}-\d{2}-\d{2}$')
    temperature_c:    Series[float] = pa.Field(ge=15.0, le=50.0)
    pluviometrie_mm:  Series[float] = pa.Field(ge=0.0)
    humidite_pct:     Series[float] = pa.Field(ge=0.0, le=100.0)

    class Config:
        strict = True
        coerce = True

def validate_and_filter(df: pd.DataFrame, schema_class) -> pd.DataFrame:
    try:
        return schema_class.validate(df, lazy=True)
    except pa.errors.SchemaErrors as exc:
        err = exc.failure_cases
        logger.warning(f'[{schema_class.__name__}] {len(err)} erreur(s) rejetées')
        valid_idx = df.index.difference(err['index'].dropna().astype(int))
        return df.loc[valid_idx]
```

2.3 Simulateur unifié Cas + Météo — kafka_producer_sentinel.py

> Le simulateur génère maintenant 2 topics : epidemio_raw (cas de maladies) ET meteo_raw (données météo). La pluviométrie est modélisée avec la saisonnalité réelle de l'hivernage sénégalais, indispensable pour le Livrable 3 (corrélation pluie/maladies).

kafka_producer_sentinel.py

```
# kafka_producer_sentinel.py — Simulateur cas sanitaires + météo
# CORRECTION : génère les 2 types de données nécessaires au Livrable 3

from kafka import KafkaProducer
import json, random, time
from datetime import date

DISTRICTS = {1:'Dakar',2:'Pikine',3:'Thiès',4:'Kaolack',
             5:'Ziguinchor',6:'Saint-Louis',7:'Tambacounda',8:'Diourbel'}
MALADIES = ['PALUD','DENGUE','CHOLERA','TYPHOIDE']
```

```
# Saisonnalité réaliste : PALUD ×3 en hivernage (juil-oct)
def weighted_maladie(month: int) -> str:
    w = [0.60,0.20,0.10,0.10] if 7<=month<=10 else [0.30,0.30,0.20,0.20]
    return random.choices(MALADIES, weights=w, k=1)[0]

# Pluviométrie corrélée à la saison (mm/jour)
def pluviometrie_realiste(month: int) -> float:
    # Hivernage sénégalais : pic en août (300mm/mois = 10mm/jour)
    pluie_moy = {1:0,2:0,3:0,4:0,5:2,6:15,7:80,8:200,9:150,10:40,11:5,12:0}
    base = pluie_moy.get(month, 0)
    return max(0.0, round(random.gauss(base/30, 2), 1))

producer = KafkaProducer(
    bootstrap_servers=['kafka:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

def gen_cas(district_id: int, d: date) -> dict:
    maladie = weighted_maladie(d.month)
    cas = random.randint(1, 80)
    return {
        'patient_raw_id':   f'SN{district_id:03d}{random.randint(100000,999999)}',
        'district_id':      district_id,
        'maladie_code':     maladie,
        'nombre_cas':       cas,
        'nombre_deces':     random.randint(0, max(1, cas//20)),
        'date_declaration': str(d),
        'source_rapport':   random.choice(['DISTRICT','HOPITAL','API'])
    }

def gen_meteo(district_id: int, d: date) -> dict:
    pluie = pluviometrie_realiste(d.month)
    return {
        'district_id':     district_id,
        'date_mesure':     str(d),
        'temperature_c':   round(random.uniform(22, 38), 1),
        'pluviometrie_mm': pluie,
        'humidite_pct':    round(min(100, 40 + pluie * 2 + random.gauss(0,5)), 1)
    }

if __name__ == '__main__':
    print('Simulateur Sentinel-SN démarré...')
    today = date.today()
    while True:
        for did in DISTRICTS:
            # Topic épidémio_raw : cas de maladies
            producer.send('epidemio_raw', gen_cas(did, today))
            # Topic meteo_raw : données météo (nécessaire pour Livrable 3)
            producer.send('meteo_raw', gen_meteo(did, today))
        producer.flush()
        print(f'[{today}] Batch envoyé — {len(DISTRICTS)*2} messages')
        time.sleep(5)
```

2.4 Pipeline Spark Streaming — streaming_health.py

1) Les 2 flux (cas + météo) ont maintenant le même watermark de 10 minutes — requis pour une jointure stream-stream valide. 2) Condition BETWEEN sur les timestamps pour borner la fenêtre de jointure. 3) COALESCE sur tous les champs nullables après la jointure LEFT.

streaming_health.py

```
# streaming_health.py — Pipeline Spark Streaming Sentinel-SN (CORRIGÉ)
# CORRECTIONS :
#   1. Watermark identique sur les 2 flux avant jointure stream-stream
#   2. Gestion explicite des nulls après jointures
#   3. Jointure stream-stream correcte avec intervalle temporel

import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, sha2, concat, lit, from_json, to_json, struct,
    window, count, avg, stddev, when, current_timestamp,
    coalesce
)
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, FloatType

SALT    = os.environ.get('HEALTH_SECRET_SALT', 'default_salt')
BROKERS = 'kafka:9092'

spark = SparkSession.builder \
    .appName('Sentinel_SN_Streaming') \
    .config('spark.sql.shuffle.partitions', '4') \
    .config('spark.jars.packages',
            'org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.0') \
    .getOrCreate()
spark.sparkContext.setLogLevel('WARN')

cas_schema = StructType([
    StructField('patient_raw_id',   StringType(),  True),
    StructField('district_id',      IntegerType(), True),
    StructField('maladie_code',     StringType(),  True),
    StructField('nombre_cas',       IntegerType(), True),
    StructField('nombre_deces',     IntegerType(), True),
    StructField('date_declaration', StringType(),  True),
    StructField('source_rapport',   StringType(),  True),
])

meteo_schema = StructType([
    StructField('district_id',     IntegerType(), True),
    StructField('date_mesure',     StringType(),  True),
    StructField('temperature_c',   FloatType(),   True),
    StructField('pluviometrie_mm', FloatType(),   True),
    StructField('humidite_pct',    FloatType(),   True),
])

# Table population (broadcast statique — pas de jointure stream/stream ici)
pop_data = [(1,'Dakar',3732000),(2,'Pikine',1462000),(3,'Thiès',817000),
            (4,'Kaolack',565000),(5,'Ziguinchor',549000),(6,'Saint-Louis',455000),
            (7,'Tambacounda',411000),(8,'Diourbel',739000)]
pop_df = spark.createDataFrame(pop_data, ['district_id','district_nom','population'])

# ── Lecture des 2 topics ──────────────────────────────────────────────
def read_kafka(topic, schema):
    return (spark.readStream
        .format('kafka')
        .option('kafka.bootstrap.servers', BROKERS)
        .option('subscribe', topic)
        .option('startingOffsets', 'latest')
        .load()
        .select(from_json(col('value').cast('string'), schema).alias('d'))
        .select('d.*')
        .withColumn('event_ts', current_timestamp())
    )
```

```
cas_df   = read_kafka('epidemio_raw', cas_schema)
meteo_df = read_kafka('meteo_raw',    meteo_schema)

# ── Privacy Layer ──────────────────────────────────────────────────────
secure_cas = (cas_df
    .withColumn('patient_id_secure',
                sha2(concat(col('patient_raw_id'), lit(SALT)), 256))
    .drop('patient_raw_id')   # Suppression définitive du PII
)

# ── CORRECTION : jointure stream-stream nécessite watermark identique ────
# Les 2 flux doivent avoir le même watermark ET une condition d'intervalle.
# Ici on joint sur district_id dans une fenêtre de 10 minutes.
cas_wm   = secure_cas.withWatermark('event_ts', '10 minutes')
meteo_wm = meteo_df  .withWatermark('event_ts', '10 minutes')

joined = (cas_wm.alias('c')
    .join(meteo_wm.alias('m'),
          expr("""
              c.district_id = m.district_id
              AND c.event_ts BETWEEN m.event_ts - INTERVAL 10 MINUTES
                                 AND m.event_ts + INTERVAL 10 MINUTES
          """),
          how='left'
    )
)

# ── CORRECTION : gestion des nulls après jointure left ──────────────────
enriched = (joined
    .join(pop_df, on='district_id', how='left')
    .withColumn('pluviometrie_mm',
                coalesce(col('m.pluviometrie_mm'), lit(0.0)))  # null → 0
    .withColumn('population',
                coalesce(col('population'), lit(1)))            # null → 1 (évite /0)
    .withColumn('taux_incidence',
                (col('c.nombre_cas') / col('population') * 100000))
)

# ── Agrégation fenêtre glissante 24h ────────────────────────────────────
windowed = (enriched
    .groupBy(window('event_ts','24 hours','1 hour'),
             'district_id','district_nom','c.maladie_code')
    .agg(
        count('c.nombre_cas').alias('nb_rapports'),
        avg('taux_incidence').alias('taux_moyen'),
        stddev('taux_incidence').alias('taux_stddev'),
        avg('pluviometrie_mm').alias('pluie_moy_mm')
    )
    .withColumn('taux_stddev', coalesce(col('taux_stddev'), lit(0.0)))
    .withColumn('statut_alerte',
        when(col('taux_moyen') > col('taux_moyen') + 2*col('taux_stddev'),
             lit('ROUGE'))
        .when(col('taux_moyen') > col('taux_moyen') + col('taux_stddev'),
             lit('ORANGE'))
        .otherwise(lit('VERT'))
    )
)

# ── Écriture vers Kafka ─────────────────────────────────────────────────
query = (windowed
    .select(to_json(struct('*')).alias('value'))
    .writeStream
    .format('kafka')
    .option('kafka.bootstrap.servers', BROKERS)
    .option('topic', 'epidemio_alerts')
    .option('checkpointLocation', '/tmp/sentinel_checkpoint')
    .outputMode('update')
    .start()
)
query.awaitTermination()
```

2.5 DAG Airflow — sentinel_retrain_dag.py

dags/sentinel_retrain_dag.py

```
# dags/sentinel_retrain_dag.py — DAG Airflow Sentinel-SN

from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.operators.dummy  import DummyOperator
from airflow.utils.dates      import days_ago
from datetime import timedelta
import subprocess, logging

logger = logging.getLogger('sentinel_dag')

default_args = {
    'owner': 'seye_ahmed', 'depends_on_past': False,
    'email_on_failure': True, 'email': ['sentinel@sonatel.sn'],
    'retries': 2, 'retry_delay': timedelta(minutes=5),
}

def check_new_cases(**ctx):
    from pyhive import hive
    conn = hive.Connection(host='hive-metastore', port=10000)
    cursor = conn.cursor()
    cursor.execute("""
        SELECT COUNT(*) FROM epidemio.cas_journaliers
        WHERE date_ingestion >= DATE_SUB(CURRENT_DATE, 1)
    """)
    count = cursor.fetchone()[0]
    ctx['ti'].xcom_push(key='nb_cas', value=count)
    return 'recalculate_thresholds' if count > 0 else 'skip_retrain'

def recalculate_thresholds(**ctx):
    from pyhive import hive
    conn = hive.Connection(host='hive-metastore', port=10000)
    cursor = conn.cursor()
    cursor.execute("""
        INSERT OVERWRITE TABLE epidemio.seuils_district
        SELECT district_id, maladie_code,
               AVG(nombre_cas)                  AS seuil_moyen,
               AVG(nombre_cas)+2*STDDEV(nombre_cas) AS seuil_alerte,
               CURRENT_DATE AS date_calcul
        FROM epidemio.cas_journaliers
        WHERE date_declaration >= DATE_SUB(CURRENT_DATE, 30)
        GROUP BY district_id, maladie_code
    """)
    logger.info('Seuils recalculés')

def update_model(**ctx):
    result = subprocess.run([
        'spark-submit', '--master', 'spark://spark-master:7077',
        '/opt/airflow/models/train_isolation_forest.py',
        '--output-path', 'hdfs:///sentinel/models/isolation_forest_latest',
        '--window-days', '90'
    ], capture_output=True, text=True, timeout=1800)
    if result.returncode != 0:
        raise Exception(f'Spark job failed: {result.stderr}')
    logger.info('Modèle mis à jour')

with DAG('sentinel_sn_retrain', default_args=default_args,
         description='Réentraînement hebdomadaire Sentinel-SN',
         schedule_interval='0 2 * * 1', start_date=days_ago(1),
         catchup=False, tags=['sentinel','mlops','sante-publique']) as dag:

    start  = DummyOperator(task_id='start')
    skip   = DummyOperator(task_id='skip_retrain')
    end    = DummyOperator(task_id='end')
    t1 = BranchPythonOperator(task_id='check_new_cases',
                              python_callable=check_new_cases, provide_context=True)
    t2 = PythonOperator(task_id='recalculate_thresholds',
                        python_callable=recalculate_thresholds, provide_context=True)
```

```
    t3 = PythonOperator(task_id='update_model',
                        python_callable=update_model, provide_context=True)
    start >> t1 >> [t2, skip]
    t2 >> t3 >> end
    skip >> end
```

2.6 Tables Hive — hive_setup.sql

> ✓ **Correction v2**
> La vue vue_alertes_districts utilise LEFT JOIN (au lieu de INNER JOIN) pour ne pas perdre les districts sans seuil calculé. COALESCE protège contre les divisions par zéro et les comparaisons avec NULL.

hive_setup.sql

```
-- hive_setup.sql — Tables Hive Sentinel-SN

CREATE DATABASE IF NOT EXISTS epidemio
    COMMENT 'Base épidémiologique Sentinel-SN — UADB 2025';
USE epidemio;

CREATE TABLE IF NOT EXISTS cas_journaliers (
    patient_id_secure  STRING,
    district_id        INT,
    district_nom       STRING,
    maladie_code       STRING,
    nombre_cas         INT,
    nombre_deces       INT,
    taux_incidence     FLOAT,
    pluviometrie_mm    FLOAT,    -- Météo jointe pour corrélation
    source_rapport     STRING,
    ingestion_ts       TIMESTAMP
)
PARTITIONED BY (date_declaration STRING, maladie STRING)
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');

CREATE TABLE IF NOT EXISTS seuils_district (
    district_id    INT, maladie_code STRING,
    seuil_moyen    FLOAT, seuil_alerte FLOAT, date_calcul DATE
) STORED AS ORC;

-- CORRECTION : jointure explicite avec seuils pour éviter les nulls
CREATE OR REPLACE VIEW vue_alertes_districts AS
SELECT
    c.district_nom,
    c.maladie_code,
    SUM(c.nombre_cas)          AS total_cas_24h,
    AVG(c.taux_incidence)      AS taux_moyen,
    COALESCE(s.seuil_alerte,99999) AS seuil_alerte,  -- null-safe
    AVG(c.pluviometrie_mm)     AS pluie_moy_mm,
    CASE
        WHEN AVG(c.taux_incidence) > COALESCE(s.seuil_alerte,99999) THEN 'ROUGE'
        WHEN AVG(c.taux_incidence) > COALESCE(s.seuil_moyen,99999)  THEN 'ORANGE'
        ELSE 'VERT'
    END AS statut_alerte
FROM cas_journaliers c
LEFT JOIN seuils_district s  -- LEFT JOIN (pas INNER) pour éviter les lignes perdues
  ON c.district_id  = s.district_id
 AND c.maladie_code = s.maladie_code
WHERE c.date_declaration = CURRENT_DATE()
GROUP BY c.district_nom, c.maladie_code, s.seuil_alerte, s.seuil_moyen;
```

2.7 Analyse de Corrélation — correlation_analysis.py

correlation_analysis.py

```
# correlation_analysis.py — Corrélation pluie / maladies hydriques
# Livrable 3 — nécessite que le simulateur ait produit des données météo

import pandas as pd
import numpy  as np
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns
from pyhive import hive

conn = hive.Connection(host='hive-metastore', port=10000, database='epidemio')

df = pd.read_sql("""
    SELECT date_declaration, district_nom, maladie_code,
           SUM(nombre_cas) AS total_cas,
           AVG(pluviometrie_mm) AS pluie_mm
    FROM cas_journaliers
    WHERE maladie_code IN ('CHOLERA','TYPHOIDE')
    GROUP BY date_declaration, district_nom, maladie_code
""", conn)

df['date_declaration'] = pd.to_datetime(df['date_declaration'])
df['mois'] = df['date_declaration'].dt.month

# Corrélation de Pearson
for maladie in ['CHOLERA','TYPHOIDE']:
    sub = df[df['maladie_code'] == maladie].dropna()
    if len(sub) > 5:
        r, pval = stats.pearsonr(sub['pluie_mm'], sub['total_cas'])
        print(f'{maladie:<10} | r={r:.3f} | p={pval:.4f}',
              '*** SIGNIFICATIF' if pval<0.05 else '')

# Visualisation
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
pivot = df.groupby(['mois','maladie_code'])['total_cas'].mean().unstack()
sns.heatmap(pivot, annot=True, fmt='.0f', cmap='YlOrRd', ax=axes[0])
axes[0].set_title('Cas moyens par mois — Maladies hydriques (Sénégal)')

for m, col_ in [('CHOLERA','#E74C3C'),('TYPHOIDE','#3498DB')]:
    sub = df[df['maladie_code']==m].dropna()
    axes[1].scatter(sub['pluie_mm'], sub['total_cas'], alpha=0.6,
                    label=m, color=col_)
    if len(sub) > 2:
        coef = np.polyfit(sub['pluie_mm'], sub['total_cas'], 1)
        axes[1].plot(sub['pluie_mm'],
                     np.poly1d(coef)(sub['pluie_mm']),
                     color=col_, linestyle='--', linewidth=2)
axes[1].set_xlabel('Pluviométrie (mm)')
axes[1].set_ylabel('Nombre de cas')
axes[1].set_title('Corrélation Pluie → Maladies hydriques')
axes[1].legend()
plt.tight_layout()
plt.savefig('sentinel_correlation.png', dpi=150, bbox_inches='tight')
print('Rapport généré : sentinel_correlation.png')
```

3. Guide de Test

> ℹ️ **Objectif**
> Ces commandes permettent de vérifier que chaque composant fonctionne correctement, étape par étape, avant la soutenance.

3.1 Tester Kafka

📄 **test_kafka.sh**

```
# Vérifier que les topics ont été créés
docker exec kafka kafka-topics.sh \
  --bootstrap-server localhost:9092 --list
# Attendu : epidemio_raw, meteo_raw, epidemio_alerts

# Démarrer le simulateur
python kafka_producer_sentinel.py &

# Lire les messages en temps réel
docker exec kafka kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic epidemio_raw --from-beginning --max-messages 5

# Vérifier le topic météo
docker exec kafka kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic meteo_raw --from-beginning --max-messages 5
```

3.2 Tester NiFi

📄 **test_nifi.sh**

```
# Vérifier que NiFi est actif
curl -s http://localhost:8081/nifi/ | grep -o '<title>.*</title>'
# Attendu : <title>NiFi</title>

# Via l'API NiFi REST : lister les processor groups
curl -s http://localhost:8081/nifi-api/process-groups/root \
  | python3 -m json.tool | grep '"name"'
```

3.3 Tester HBase

📄 **test_hbase.sh**

```
# Lister les tables créées par hbase_setup.py
docker exec hbase hbase shell <<EOF
list
EOF
# Attendu : epidemio:cas, epidemio:alertes, epidemio:seuils

# Vérifier les familles de colonnes
docker exec hbase hbase shell <<EOF
describe 'epidemio:cas'
EOF
```

3.4 Tester le Pipeline Spark

test_spark.sh

```
# Soumettre le job streaming
docker exec spark-master spark-submit \
  --master spark://spark-master:7077 \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.0 \
  /opt/streaming_health.py

# Vérifier les alertes générées
docker exec kafka kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic epidemio_alerts --from-beginning --max-messages 3

# Attendu : JSON avec statut_alerte VERT/ORANGE/ROUGE
```

4. Livrables Attendus

#	Livrable	Contenu attendu
1	**Template NiFi (.xml)**	Flux GetFile → UpdateAttribute → PublishKafka pour les 3 sources, testé via interface http://localhost:8081
2	**Scripts Spark**	streaming_health.py + hbase_setup.py fonctionnels, avec jointure stream-stream et gestion des nulls
3	**Rapport de Corrélation**	Jupyter Notebook + graphiques corrélation pluie/maladies, p-value < 0.05 démontrée
4	**Dashboard d'Alerte**	Vue Hive + dashboard districts ROUGE/ORANGE/VERT avec au moins 1 alerte ROUGE générée en live

5. Barème de Notation — /20 points

> ℹ️ **Nouveauté v2**
> Le barème distingue désormais 4 niveaux de maîtrise pour chaque critère : Insuffisant (0 pt), Passable (50%), Bien (75%), Excellent (100%). Un bonus documentation de 2 pts est accordé pour un README complet et des commentaires de code.

Critère	Pts	Insuffisant (0)	Passable (50%)	Bien (75%)	Excellent (100%)
Ingestion NiFi Multi-Sources	**3 pts**	NiFi absent ou non fonctionnel	1 source sur 3 routée vers Kafka	2 sources routées, messages partiels	3 sources, messages bien formés dans les 2 topics
Validation & Privacy (Pandera/CDP)	**4 pts**	Pas de validation ni d'anonymisation	Schéma partiel OU anonymisation sans sel	Schéma complet + SHA-256 + sel, PII visible en log	Aucun PII en clair, validation lazy, erreurs loguées
Streaming & Jointures	**4 pts**	Pas de pipeline Spark fonctionnel	Lecture Kafka seule, pas de jointure	Jointures correctes, nulls non gérés	Watermarks cohérents, COALESCE, taux d'incidence calculé
MLOps & Détection + Airflow	**5 pts**	Pas de modèle ni de DAG	Règles expertes seules OU DAG sans branchement	Fenêtres glissantes + DAG avec T1→T2→T3	Isolation Forest ou K-Means + DAG conditionnel + RMSE affiché
Mirroring HBase/Hive & Dashboard	**4 pts**	Ni HBase ni Hive configurés	Tables créées mais vides	Données présentes, vue Hive fonctionnelle	Dashboard live avec alertes ROUGE visibles en soutenance
TOTAL	**20 pts**				

Bonus Documentation (jusqu'à +2 pts)

Élément	Points bonus
README.md avec instructions docker compose up	+0.5 pt
Commentaires de code en français (tous les scripts)	+0.5 pt
Tests unitaires (pytest) sur les schémas Pandera	+0.5 pt
Démo live sans erreur en soutenance	+0.5 pt

> ⚠️ **Rappel PII**
> Toute fuite de données personnelles identifiables (patient_raw_id non supprimé, noms en clair) entraîne la perte intégrale des 4 pts Privacy, sans recours possible.

Bon courage

"Always be a solution, never a problem."

Mr Ahmed Ben Sidy Bouya SEYE | Senior Big Data & AI Engineer | Groupe Sonatel